LÖSUNGEN · LEHREREXEMPLAR

W2·1 – Standardbausteine und Suchen: Lösungen

Zum Arbeitsblatt `AQ_W2_AB1_Standardbausteine_und_Suchen.docx` · IDE-Level 2.1: siehe `AQ_W2_L2_Rangliste_LSG.txt` · alle Werte mit Java nachgerechnet

Aufgabe 0 – Blitz-Warm-up

Ausdruck	`7 / 2`	`(double) 7 / 2`	`(double) (7 / 2)`	`7 / 2.0`
Wert	3	3.5	3.0	3.5

b) Belegt: Indizes 0 bis 5. In `eintraege[6]` steht `null`. Letzter gültiger Index: 5 = anzahl − 1 (nicht 9 = eintraege.length − 1). **c)** `double schnitt = (double) summe / anzahl;` – Cast **vor** der Division. `(double) (summe / anzahl)` ist falsch (erst ganzzahlig geteilt). Gesprächsanlass: Was passiert bei anzahl = 0?

Aufgabe 1 – Das Rätsel der Bestzeit (9 BE)

a) Aufruf `geheim(1000)` auf der Demo-Rangliste (3 BE):

i	Name	pos
–		−1
0	Aria (1200, 95 s)	0
1	Brann (800, 120 s)	0
2	Cyra (1500, 80 s)	2
3	Dorn (950, 110 s)	2
4	Elin (600, 140 s)	2
5	Finn (1100, 101 s)	2

Rückgabewert **2** (Cyra). **b)** `geheim(1200)` → **2** (Kandidaten Aria und Cyra, Cyra schneller); `geheim(2000)` → **−1** (kein Eintrag erfüllt die Bedingung) (1 BE).

c) Die Methode liefert den Index des Eintrags mit der **kürzesten Spielzeit unter allen Einträgen mit mindestens grenze Punkten**; gibt es keinen solchen Eintrag, −1. Bei gleicher Zeit gewinnt der erste (strikt <). Name z. B. `indexSchnellsterAb(grenze)` oder `sonderpreis(grenze)`. Kombiniert: **Minimum mit Index** (über die Zeit) und eine **Bedingung/Filter** wie beim Zählen; der Startwert −1 übernimmt die Rolle der Leer-Abfrage der linearen Suche (3 BE).

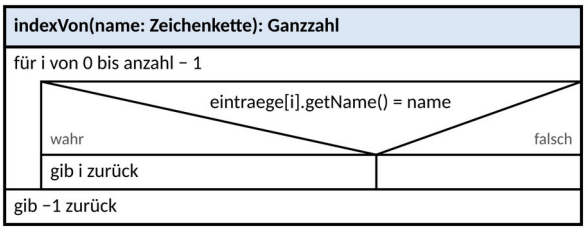
d) Java wertet `||` von links nach rechts mit **Kurzschluss** aus: Ist `pos == -1` wahr, wird der rechte Teil nicht mehr ausgewertet. Stünde er vorne, würde beim ersten Kandidaten `eintraege[-1]` ausgewertet → ArrayIndexOutOfBoundsException (2 BE).

Aufgabe 2 – Level 2.1, Teil 1

Lösungen siehe `AQ_W2_L2_Rangliste_LSG.txt`. Häufig: `hoechstePunktzahl` mit `max = 0` (Test `experte_negativePunkte` rot), `>=` bei besterEintrag (Test `experte_besterEintrag_gleichstand` rot), `>=` bei anzahlUeber (`erweiterung_anzahlUeber_grenzeGenau` rot).

Aufgabe 3 – Papier-Compiler indexVon

```
public int indexVon(String name) {
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getName().equals(name)) {
            return i;
        }
    }
    return -1;
}
```



Bewertung als Abi-Aufgabe (4 BE): Schleife über 0 … anzahl − 1 (1) · Vergleich mit equals (1) · sofortige Rückgabe des ersten Treffers (1) · −1 nach der Schleife (1). **Typische Fehler:** `return -1` im else (Suche endet nach Index 0) · `==` statt equals (Test `experte_indexVonMitEquals` rot) · Schleife bis `eintraege.length` → NullPointerException bei `null.getName()` · `name.equals(eintraege[i])` (vergleicht Zeichenkette mit Eintrag, immer false).

Aufgabe 4 – Trace mit Doppelschleife (8 BE)

Vergleich Nr.	i	j	eintraege[i].getPunkte()	eintraege[j].getPunkte()
1	0	1	700	900
2	0	2	700	650
3	0	3	700	1100
4	0	4	700	900
5	1	2	900	650
6	1	3	900	1100
7	1	4	900	900

Rückgabewert **wahr** nach **7** Vergleichen: In Zeile 7 sind beide Punktzahlen gleich (900), die Methode gibt sofort wahr zurück (3 BE). **b)** 5 Einträge ohne Duplikat: 4 + 3 + 2 + 1 = **10** Vergleiche; 6 Einträge: 5 + 4 + 3 + 2 + 1 = **15**. Vermutung: (n − 1) + (n − 2) + … + 1 = **n(n − 1)/2** (Herleitung folgt in W2·3) (3 BE). **c)** Ab 0 würde jeder Eintrag auch mit sich selbst verglichen (i = j → immer gleich → Methode liefert **immer** wahr), außerdem würde jedes Paar doppelt geprüft (2 BE).

Aufgabe 5 – Bug-Hunt (9 BE)

Methode	Zeile	Korrektur
A	5 (und 6)	`int max = eintraege[0].getPunkte();` – Schleife dann ab `i = 1` (ab 0 wäre auch korrekt, nur ein Vergleich mehr)
B	3	`if (eintraege[i].getName().equals(name)) {`
C	3	`for (int i = 0; i < anzahl; i++) {`

b) z. B. Strafpunkte: Brann −50, Dorn −20, Elin −80. Methode A liefert **0** (ein Wert, den es gar nicht gibt), richtig wäre **−20** (2 BE).

c) `==` vergleicht bei Objekten (also auch Zeichenketten) in echtem Java die **Referenzen**. Wird der gesuchte Name zur Laufzeit gebaut (z. B. `teil + "ra"` mit `String teil = "Cy";` oder eine Benutzereingabe),

ist es ein anderes Objekt → Ergebnis −1, obwohl Cyra an Index 2 steht (mit Java nachgeprüft). Die Online-IDE vergleicht mit `==` den Inhalt – darum fällt es dort nicht auf, im Abitur kostet es aber die BE (2 BE).

d) Demo-Rangliste: Bei i = 6 = anzahl ist `eintraege[6]` null → `getPunkte()` auf null → **NullPointerException**, Abbruch. Volle Rangliste: Index anzahl = eintraege.length existiert nicht → **ArrayIndexOutOfBoundsException**. In keinem Fall wird ein Ergebnis zurückgegeben (2 BE).

Aufgabe 6 – Level 2.1, Teil 2

c) Ausgaben des Hauptprogramms: Ueber 1000 Punkte: **3** (Aria, Cyra, Finn) · Index von Dorn: **3** · Durchschnitt: **1025.0** (6150 / 6). **Denkanstoß:** 1000 Einträge ohne Duplikat: 999 + 998 + … + 1 = 1000 · 999 / 2 = **499 500** Vergleiche.

Für Schnelle – Die Siegerehrung (mögliche Lösungen)

Demo-Liste: platzVon("Dorn") = **4**, platzVon("Cyra") = 1, anzahlUeberDurchschnitt() = **3**, zweitbesterEintrag() = **Aria**. Bei Punktgleichstand erhalten beide denselben Platz (Platz 1, 1, 3 – wie im Sport). Häufiger Fehler bei zweitbester: Beim neuen Ersten wird der alte Erste vergessen statt auf Platz 2 geschoben.

```
public int platzVon(String name) {
    int pos = indexVon(name);
    if (pos == -1) {
        return -1;
    }
    int besser = 0;
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getPunkte() > eintraege[pos].getPunkte()) {
            besser++;
        }
    }
    return besser + 1;
}

public int anzahlUeberDurchschnitt() {
    double schnitt = durchschnitt();
    int zaehler = 0;
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getPunkte() > schnitt) {
            zaehler++;
        }
    }
    return zaehler;
}
```

```
public Eintrag zweitbesterEintrag() {
    if (anzahl < 2) {
        return null;
    }
    Eintrag erster = eintraege[0];
    Eintrag zweiter = eintraege[1];
    if (zweiter.getPunkte() > erster.getPunkte()) {
        erster = eintraege[1];
        zweiter = eintraege[0];
    }
    for (int i = 2; i < anzahl; i++) {
        Eintrag e = eintraege[i];
        if (e.getPunkte() > erster.getPunkte()) {
            zweiter = erster;           // alter Erster rutscht auf Platz 2
            erster = e;
        } else if (e.getPunkte() > zweiter.getPunkte()) {
            zweiter = e;
        }
    }
    return zweiter;
}
```